DX Audit Report

Version 1.0

Published by Dionis Xhaferi

5/9/25

Presale Smart Contract Audit Report

Protocol Summary

The presale protocol is a Solana smart contract designed to manage a token sale where users can purchase tokens using USDC.The protocol enforces a hard cap on total tokens sold and limits the number of tokens each wallet can purchase. Upon initialization, the owner sets the initial token price, lockup period, and sale parameters. Buyers transfer USDC directly to a designated treasury account, and their purchase amounts are recorded. After the lockup period ends and the owner enables claiming, users can claim their tokens from a vault account controlled by a program-derived address. The protocol also allows the owner to end the sale or activate the claim phase.

Disclaimer

The DX team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Rust implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Executive Summary

Issues found

Severity	**Number of issues found**
High	2
Medium	2
Low	1
Info	2
Total	7

Findings

High

[H-1] Missing Access Control for OnlyOwner Context

Description: The OnlyOwner context allows any signer, without checking if it's the actual owner.

Impact: High - Unauthorized calls to enable claim or end presale.

Proof of Concept:

```
pub struct OnlyOwner<'info> { pub authority: Signer<'info>, }
```

Recommended Mitigation:

```
Store the owner in state and use has_one validation.
```

[H-2] No Cap on desired_amount Input in buy_tokens

Description: desired_amount is unchecked, opening risk for overflow logic.

Impact: High - Can be abused for gas griefing or simulate overflow.

Proof of Concept:

```
pub fn buy_tokens(ctx: Context<BuyTokens>, desired_amount: u64) -> Result<()> {
```

Recommended Mitigation:

```
Add an upper bound to desired_amount input.
```

Medium

[M-1] wallet_purchases Uses In-Memory BTreeMap

Description: Anchor does not persist BTreeMap, buyers' balances reset.

Impact: Medium - Users can bypass wallet limits repeatedly.

Proof of Concept:

```
pub wallet_purchases: std::collections::BTreeMap<Pubkey, u64>
```

Recommended Mitigation:

```
Use PDA-based user storage or Account for each wallet.
```

[M-2] vault_authority Is Not Verified in claim_tokens

Description: vault_authority is CHECK without runtime validation.

Impact: Medium - Risk of spoofed authority and faulty transfers.

Proof of Concept:

```
pub vault_authority: AccountInfo<'info>
```

Recommended Mitigation:

```
Use find_program_address to verify PDA.
```

Low

[L-1] seeds in claim_tokens Are Incomplete

Description: seeds used for signer do not include bump seed.

Impact: Low - PDA transfer authorization may fail.

Proof of Concept:

```
let seeds = &[b"authority".as_ref()];
```

Recommended Mitigation:

```
Include bump from find_program_address in seeds.
```

Informational

[I-1] Magic Numbers for Tiers

Description: Tier size hardcoded multiple times, limits flexibility.

Impact: Informational - Makes updates harder.

Proof of Concept:

```
let tier_index = state.total_sold / 5_000_000;
```

Recommended Mitigation:

```
Declare tier size as constant.
```

[I-2] Precision Loss in Tier Price Calculation

Description: Using f64 for math may introduce rounding errors.

Impact: Informational - Could cause pricing inaccuracy in edge cases.

Proof of Concept:

```
let tier_price = ((state.initial_price as f64) * 1.28f64.powf(tier_index as f64)) as u64;
```

Recommended Mitigation:

```
Consider fixed-point math or rust_decimal.
```